Hitesh Rathore (HR) Notes

in https://www.linkedin.com/in/hitesh-rathore-835279305/

JS

NUMBER IN JAVASCRIPT

FIRST PRINCIPLES APPROACH

What is a Number in JavaScript?

1 STEP 1

Before understanding a Number, we must first understand:

What is a Value?

Examples:

`10` `20` `true` `"hello"` `null` `undefined`

All of these are values.

A value is simply:
A piece of data that exists in memory.

2 STEP 2

JavaScript divides values into two major categories.

1 PRIMITIVE VALUES

2 OBJECTS

PRIMITIVE VALUES

- 10
- "hello"
- true
- undefined
- null
- Symbol()
- 10n

These are the smallest and simplest values JavaScript can work with.

OBJECTS

- {}
- []
- new Date()
- new Number(10)

Objects are collections of:

Data + Behavior

3 What Does "Primitive" Mean?

Primitive means:

- A value that is not an object.
- A value that directly contains its actual data.

Think:

Primitive = Raw Data

EXAMPLE

```
let age = 25;
```

age
25

- The variable directly contains: 25
- There is no object around it.
- There is no property table.
- There is no method storage.

Just: 25

4 What Is A Number Primitive?

A Number Primitive is simply:

A raw numeric value stored directly as a primitive.

EXAMPLES

- 10
- 20
- 3.14
- -5
- Infinity
- NaN

All of these are: **Number Primitives** ☆ ☆ ☆

5 MEMORY VISUALIZATION

Suppose:

```
let x = 10;
```

Conceptually:

VARIABLE TABLE
x
10

→ Or

x
10

The value itself is stored.
There is no Number Object.

6

```
let price = 99.99;
```

Conceptually:

price
99.99

Again: Raw Number Value

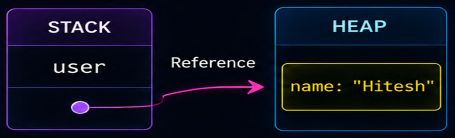
✓ No Object

7

Consider:

```
let user = { name: "Hitesh" };
```

Conceptually:



- The variable does not directly contain the object.
- It contains a reference to the object.
- It stores a reference.

8 NUMBER PRIMITIVES ARE IMMUTABLE

Suppose:

```
let x = 10;
```

Later:

```
x = 20;
```

?

Question: Did JavaScript modify the old value 10?

ANSWER

No

- The old primitive remains unchanged.
- The variable simply starts holding a different value.

Before:		After:
x		x
10	→	20

9 HOW DOES THE ENGINE STORE IT?

The JavaScript Specification says: Number

Most JavaScript engines internally store it as:

IEEE-754 Double Precision Floating Point Format

Conceptually:

10 → Number Primitive → IEEE-754 Double Precision

ⓘ Even integers use this format.

10 NUMBER PRIMITIVE VS NUMBER OBJECT

These are NOT the same thing.

10 VS **new Number(10)**

Primitive | Object

11 MOST NUMBERS IN JAVASCRIPT ARE PRIMITIVES

When you write:

5 | 10 | 20 | 3.14 | ...

↓

They are all: **Number Primitives**

✗ Not Number Objects ✗

NUMBER PRIMITIVE

```
let x = 10;
```

Visualization:

x

10

Type: `typeof x`

Output: `"number"`

VS

NUMBER OBJECT

```
let y = new Number(10);
```

Visualization:

y → Reference → `[[NumberData]]: 10`

Type: `typeof y`

Output: `"object"`

MENTAL MODEL

10, 3.14, 99, -5, Infinity, NaN

↓

Raw Numeric Value

↓

Not An Object

↓

Stored Internally As IEEE-754 Double Precision

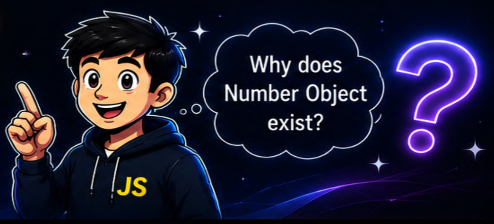
↓

typeof === “number”

NUMBER OBJECT IN JAVASCRIPT

JS

★ FIRST PRINCIPLES APPROACH ★


JS
Why does Number Object exist?

1

Before learning what a Number Object is, we must first understand:

Why does it exist at all?

2 START WITH A NUMBER PRIMITIVE

Suppose:

```
let x = 10;
```

What is 10?

It is a: Number Primitive

Conceptually:

x → 10

- Just a raw numeric value.
- No Properties
- No Methods
- No Object Identity

10

2 A PROBLEM APPEARS

Suppose we write:

```
let x = 10;
x.toString();
```

?

Question: How can 10 call a method?

- Because we just learned 10 is NOT an Object
- And methods belong to Objects.

CONTRADICTION

10

↓

Primitive

But

`10.toString()`

↓

Works

How can both be true?

3 JAVASCRIPT NEEDS A SOLUTION

JavaScript solved this challenge:

GOAL 1

Keep numbers lightweight

Example:

```
let age = 25;
```

Works exactly as intended.

✓

GOAL 2

Allow useful methods.

- `age.toString()`
- `age.toFixed()`
- `age.toPrecision()`

Flexible and powerful.

✓

QUESTION

?

How can both goals be achieved?

ANSWER

Number Objects!

4 THE NUMBER OBJECT IDEA

JavaScript introduces **Number Object.**

Think of it as a box that wraps the primitive `value.`

Conceptually:

10

↓ Number Object

`[[NumberData]] : 10`

This box is called: **Number Object.**

5 CREATING A NUMBER OBJECT EXPLICITLY

You can create one manually!

Example:

```
let n = new Number(10);
```

Conceptually:

n → `[[NumberData]] : 10`

Here, `n` is a real `Object`!

CHECK FOR TYPE

`typeof n`

↓

`"object"`

Because `n` is an `Object`, not a primitive!

6 INTERNAL STRUCTURE

`let n = new Number(10);`

Creates something similar to:

Number Object

- `[[NumberData]] : 10`
- `toString()`
- `toFixed()`
- `toPrecision()`
- `valueOf()`

Important:
`[[NumberData]]`
is an Internal Slot.

It stores the actual numeric value.

7 NUMBER PRIMITIVE vs NUMBER OBJECT

NUMBER PRIMITIVE

`let x = 10;`

Conceptually:

x → 10

Type: `typeof x`

Output:

`"number"`

VS

NUMBER OBJECT

`let y = new Number(10);`

Conceptually:

y → `[[NumberData]] : 10`

Type: `typeof y`

Output:

`"object"`

8 COMPARISON DIFFERENCE

Primitive Values

`10 === 10`

↓

Output:

`true`

Because JavaScript compares values.

NUMBER OBJECTS

`new Number(10) === new Number(10)`

↓

Output:

`false`

Why? Because:
Object #1 ≠ Object #2
Even if both contain: 10

9 VISUALIZATION

Object A

`[[NumberData]] : 10`

Object B

`[[NumberData]] : 10`

Different Objects
↓
Different Identity
↓
Comparison Returns False

10

A Number Object can reveal the primitive value inside it.

`[[NumberData]] : 10`
↓ valueOf()
10

Example:

```
let n = new Number(10);
n.valueOf();
```

Output:

10

11 WHY WE RARELY USE NUMBER OBJECTS

Primitive Version

`let a = 10;`

- ✓ Simple
- ✓ Fast
- ✓ Memory Efficient

Now compare with:

`let a = new Number(10);`

Requires:

- Object Creation
- Memory Allocation
- Reference Handling
- Garbage Collection

For almost no practical benefit.

Therefore:

Most JavaScript code uses: `10` Not: `new Number(10)`

11 THEN WHY DOES THIS WORK?

`10 .toString()` →

If 10 is not an Object, how can it use methods?

ANSWER
AUTO BOXING

JavaScript temporarily creates a hidden Number Object.

10 → Temporary Number Object `[[NumberData]] : 10` → toString() Executed → Result Produced → Temporary Object Destroyed

↓

Primitive Value

IMPORTANT

Method Exists On Number Object

But The Value Remains A Primitive

WHAT HAPPENS INTERNALLY WHEN WE WRITE new Number(10)

★ FIRST PRINCIPLES APPROACH ★

JS
Let's see what really happens!

1 WHAT DOES "new" MEAN?

Whenever JavaScript sees:

`new Something(...)`

It understands:

Create a brand new Object.

Therefore:

`new Number(10)`

does NOT mean create a Number Primitive.

Instead it means:

Create a Number Object.

2 BEFORE EXECUTION

Code:

`let n = new Number(10);`

Conceptual Memory

Stack	Heap
n	(empty)

At this moment:
No Number Object exists.

3 JAVASCRIPT CREATES A NEW EMPTY OBJECT

`new Number(10)` starts with:

Create an Empty Object

Heap
Object #1

The object exists, but currently contains nothing.

4 ATTACH Number.prototype

The new object must behave like a **Number Object.**

Therefore JavaScript links it to:

`Number.prototype`

Object #1 ↓ `Number.prototype`

Provides methods:

- `toString()`
- `toFixed()`
- `toPrecision()`
- `valueOf()`

The object does not own these methods.
It inherits them from Number.prototype.

5 CREATE AN INTERNAL SLOT

JavaScript now creates a hidden internal slot: `[[NumberData]]`

Object #1

`[[NumberData]] : ?`

- 🔒 This slot is special.
- 🔓 Normal objects do not have it.
- 🎯 It exists specifically for Number Objects.

6 CONVERT THE ARGUMENT

Argument Passed:

10

JavaScript converts it into a Number value.

↓

10

Number Primitive

↓

10

Nothing changes because the value is already a valid Number. ✓

7 STORE THE VALUE INSIDE `[[NumberData]]`

JavaScript places the primitive value inside the hidden slot.

Object #1

`[[NumberData]] : 10`

- ★ This is the most important step.
- ☆ The object itself is NOT 10.
- 🎯 The object CONTAINS 10.

THINK OF IT LIKE THIS

Number Primitive

↓

10

↓

Wrapped Inside

↓

Number Object

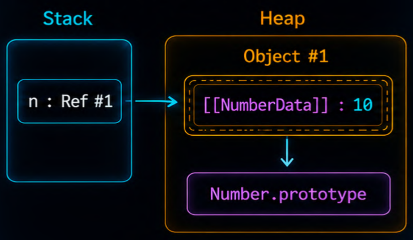
`[[NumberData]] : 10`

8 RETURN THE OBJECT REFERENCE

The constructor finishes execution.

It returns: `Reference To Object #1`

Memory Now Looks Like:



Stack

`n : Ref #1` →

Heap

Object #1

`[[NumberData]] : 10`

↓

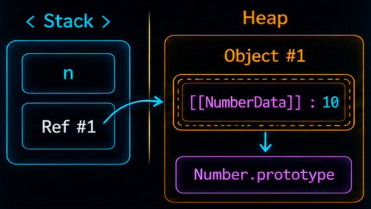
`Number.prototype`

9 FINAL STATE

Code:

```
let n = new Number(10);
```

Conceptually:



< Stack >

n

Ref #1

Heap

Object #1

`[[NumberData]] : 10`

↓

`Number.prototype`

ⓘ The variable stores a Reference, not the actual Number.

10 WHY DOES typeof RETURN "object" ?

Suppose: `typeof n`

JavaScript sees:

- n - → Reference

↓

Object #1

`[[NumberData]] : 10`

`typeof n`

↓

"object"

✓

Since the value refers to an Object, the result becomes: "object"

11 HOW DOES valueOf() WORK?

Suppose: `n.valueOf()`

JavaScript finds valueOf() in `Number.prototype`.

The method reads `[[NumberData]]`.

Object #1

`[[NumberData]] : 10`

↓

Output: 10

12 HOW DOES toString() WORK?

Suppose: `n.toString()`

Internally:

Read `[[NumberData]]`

↓

10

↓

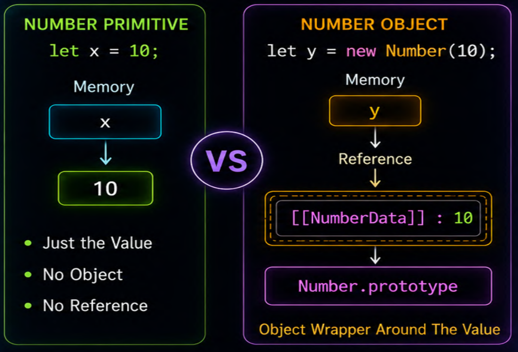
Convert Number To String

↓

"10"

Output: "10"

13 NUMBER PRIMITIVE vs NUMBER OBJECT



NUMBER PRIMITIVE

`let x = 10;`

Memory

x

↓

10

- Just the Value
- No Object
- No Reference

VS

NUMBER OBJECT

`let y = new Number(10);`

Memory

y

↓

Reference

↓

`[[NumberData]] : 10`

↓

`Number.prototype`

Object Wrapper Around The Value

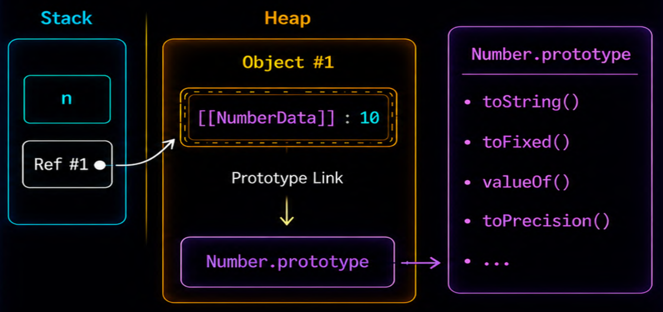
14 INTERNAL SUMMARY

When JavaScript executes: `new Number(10)`

Create Empty Object → Link Object To `Number.prototype` → Create `[[NumberData]]` Internal Slot → Store Primitive Value 10 Inside `[[NumberData]]` → Return Reference To The Object

15 FINAL MENTAL MODEL

```
let n = new Number(10);
```



Stack

n

Ref #1 ●

Heap

Object #1

`[[NumberData]] : 10`

Prototype Link

↓

`Number.prototype` →

`Number.prototype`

- `toString()`
- `toFixed()`
- `valueOf()`
- `toPrecision()`
- `...`

INTERNAL STRUCTURE OF A NUMBER OBJECT

FIRST PRINCIPLES APPROACH

1 REMEMBER: A NUMBER OBJECT IS NOT THE NUMBER ITSELF

When you write:

```
let n = new Number(10);
```

Many beginners imagine:

n
↓
10

❌ This is wrong.

That is a Number Primitive.

What actually exists is:

n
↓
Number Object
↓
[[NumberData]] : 10

The object is a Wrapper.

The actual numeric value lives inside a hidden Internal Slot.

2 WHAT IS AN INTERNAL SLOT?

JavaScript objects can contain hidden data that programmers cannot directly access.

The ECMAScript Specification calls these:

`Internal Slots`

They are written using:

`[[Something]]`

Examples:

- [[Prototype]]
- [[Extensible]]
- [[NumberData]]
- [[StringData]]
- [[DateValue]]

3 NUMBER OBJECT CONTAINS [[NumberData]]

Suppose:

```
let n = new Number(10);
```

Conceptually:

Heap
Object #1
[[NumberData]] : 10

10

The slot stores:

The Wrapped Number Primitive

In this example: 10

4 WHY DOES [[NumberData]] EXIST?

Imagine Number Objects looked like:

```
value : 10
```

Then programmers could do:

```
n.value = 500;
```

and accidentally change data that represents the object's numeric value.

JavaScript wanted a protected location.

Therefore:

`[[NumberData]]`

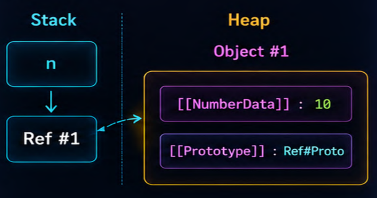
is hidden and controlled only by the engine.

5 MEMORY DIAGRAM

Code:

```
let n = new Number(10);
```

Conceptually:



ⓘ Notice: 10 is stored INSIDE the Object.

6 THE VALUE IS STILL A PRIMITIVE

☆ **Very Important**

Inside:

`[[NumberData]]`

{ } JavaScript stores:

A Number Primitive

NOT another Number Object.

Visualization

Number Object

[[NumberData]]
↓
10
↓
Object contains Primitive

7 WHAT DOES valueOf() DO?

Example:

```
let n = new Number(10);
n.valueOf();
```

Internally:

Read [[NumberData]]

Memory

[[NumberData]] : 10
↓
10

Output:

10

Mental Flow

valueOf()
↓
Read Wrapped Primitive
↓
Return Primitive
↓
10

8 WHAT DOES toString() DO?

Example:

```
n.toString();
```

Internally:

Read
[[NumberData]]
↓
10
↓
Convert Number To String
↓
"10"

Output:

"10"

☆ **IMPORTANT**

Most Number methods do **NOT** operate on the Object itself.

They operate on:

`[[NumberData]]`

The primitive value stored inside the object.

9 MORE DETAILED STRUCTURE

Conceptually:

```
let n = new Number(10);
```

Can be imagined as:

Object #1
[[NumberData]] : 10
[[Prototype]] : Number.prototype

PROTOTYPE CHAIN

Object #1 → Number.prototype → Object.prototype
↓
null

This chain allows the object to access inherited methods.

10 WHY NUMBER METHODS WORK

Example:

```
n.toFixed(2);
```

JavaScript searches:

Object #1
Method Not Found
Number.prototype
Method Found
Execute Method
Read [[NumberData]]
Use Value 10
Return "10.00"
↓
"10.00"

Conceptual Flow

Object #1
↓
Number.prototype toFixed()
↓
Read [[NumberData]]
↓
10
↓
Return "10.00"

11 NUMBER PRIMITIVE VS VS NUMBER OBJECT

NUMBER PRIMITIVE

```
let x = 10;
```

Conceptually:

x
↓
10

- Nothing Else
- No Internal Structure
- No Prototype Link
- No Wrapper Object

NUMBER OBJECT

```
let y = new Number(10);
```

Conceptually:

y
↓
[[NumberData]] : 10
[[Prototype]] : Ref

- More Memory
- More Structure
- More Overhead

12 AUTO BOXING CONNECTION

When you write:

```
(10).toString()
```

JavaScript temporarily creates something conceptually similar to:

Temporary Number Object

[[NumberData]] : 10
↓
Call toString()
↓
Read [[NumberData]]
↓
Convert To String
↓
Return Result
↓
"10"

After that:

Temporary Object is discarded.

THIS IS WHY

Primitives can use methods even though Primitives are **NOT** Objects.

13 ONE-LINE DEFINITION

[[NumberData]] is a hidden internal slot inside a Number Object that stores the actual wrapped Number Primitive value, and most Number methods operate on this internal value rather than on the Object itself.

JS WHY Number.prototype EXISTS?

FROM FIRST PRINCIPLES

PART 1 THE WORLD WITHOUT Number.prototype

Imagine JavaScript designers create Number Objects.

```
let a = new Number(10);
let b = new Number(20);
let c = new Number(30);
```

Conceptually:

a → Number Object A
b → Number Object B
c → Number Object C

Now we want:

```
a.toString()
b.toString()
c.toString()
```

to work.

Question:

Where should JavaScript store `toString()`?

POSSIBILITY 1

Store it inside every object.

Looks okay?
Actually this is terrible.

Number Object A	Number Object B	Number Object C
[[NumberData]] : 10	[[NumberData]] : 20	[[NumberData]] : 30
toString()	toString()	toString()
toFixed()	toFixed()	toFixed()
toPrecision()	toPrecision()	toPrecision()
valueOf()	valueOf()	valueOf()

PROBLEM

Suppose:

```
let n1 = new Number(10);
let n2 = new Number(20);
let n3 = new Number(30);
```

Now imagine:

1000
Number Objects

Each object contains:

- toString() again
- toFixed() again
- toPrecision() again
- valueOf() again

1000 times

Huge memory waste.

PART 2 JAVASCRIPT'S IDEA

Object A	Object B	Object C
[[NumberData]] : 10	[[NumberData]] : 20	[[NumberData]] : 30

Conceptually:

JavaScript creates ONE shared object.

Number.prototype

Conceptually:

Number.prototype
- toString()
- toFixed()
- toPrecision()
- valueOf()

Only one copy exists.

PART 3 CONNECTING OBJECTS

Now JavaScript says:

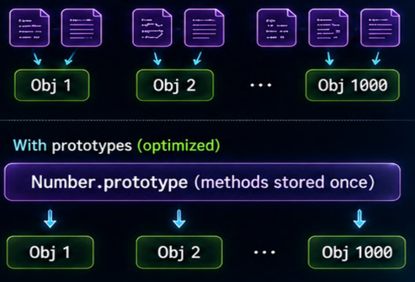
Object A	Object B	Object C
[[NumberData]] : 10	[[NumberData]] : 20	[[NumberData]] : 30
↓	↓	↓
Number.prototype	Number.prototype	Number.prototype

✓ Every Number Object shares the same methods.

✓ Memory usage becomes:

Methods stored once
instead of
Methods stored 1000 times

Huge optimization.


Without prototypes (wasteful)

With prototypes (optimized)

4 Let's see how will this work

Suppose:

```
let n = new Number(10);
```

↓

```
n.toString();
```

↓

Question:

toString is NOT inside n.

So how does JavaScript find it?

JavaScript performs a search.

n
↓
Object #1
[[NumberData]] : 10
↓
Search: toString ?
↓
Not found. ✕

Then engine says:

Let's follow the prototype link.

↓
Object #1
↓
Number.prototype

Search again.

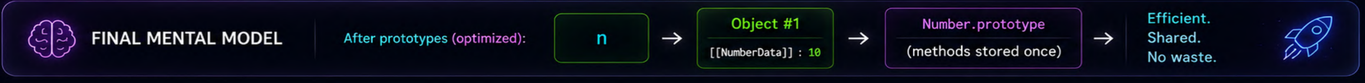
toString ?
↓
Found. ✓
↓
Execute it.

THIS IS THE ACTUAL PURPOSE OF Number.prototype

- ★ Not because JavaScript wanted a fancy feature.
- ★ Not because OOP needs prototypes.

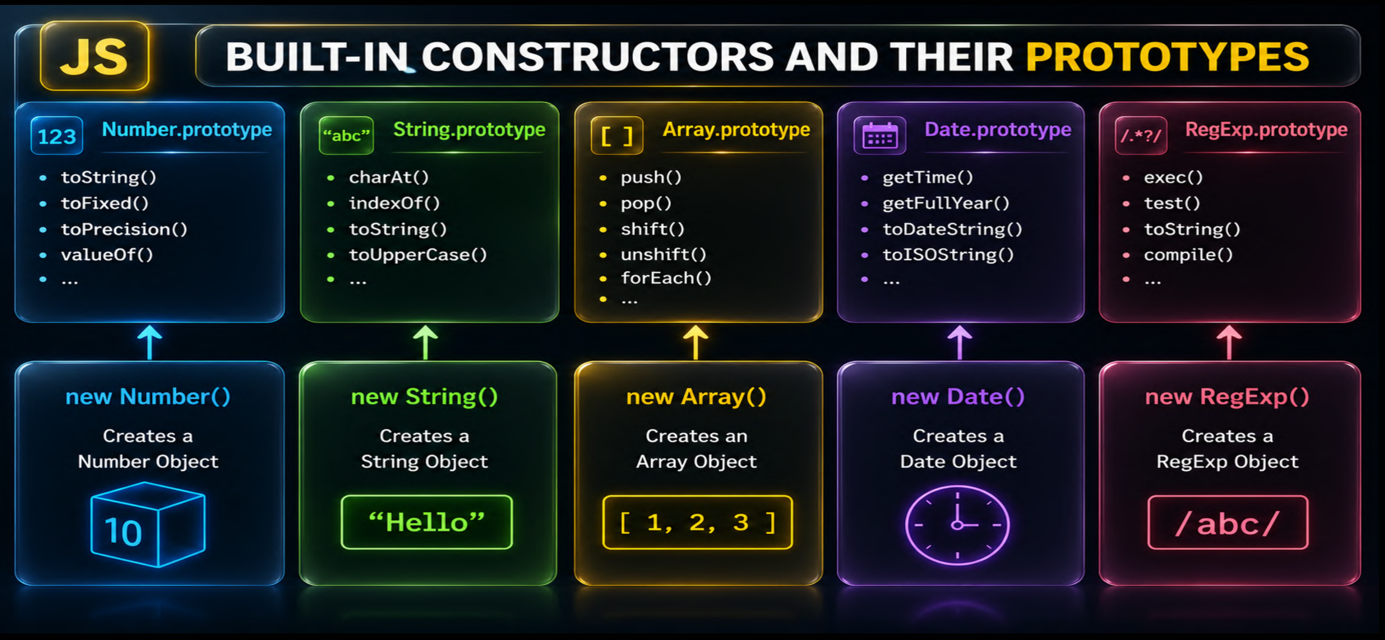
The original problem was:

Thousands of Number Objects
↓
Thousands of copies of the same methods
↓
Huge memory waste
↓
Need one shared location
↓
Create Number.prototype
↓
Store methods once
↓
All Number Objects share them



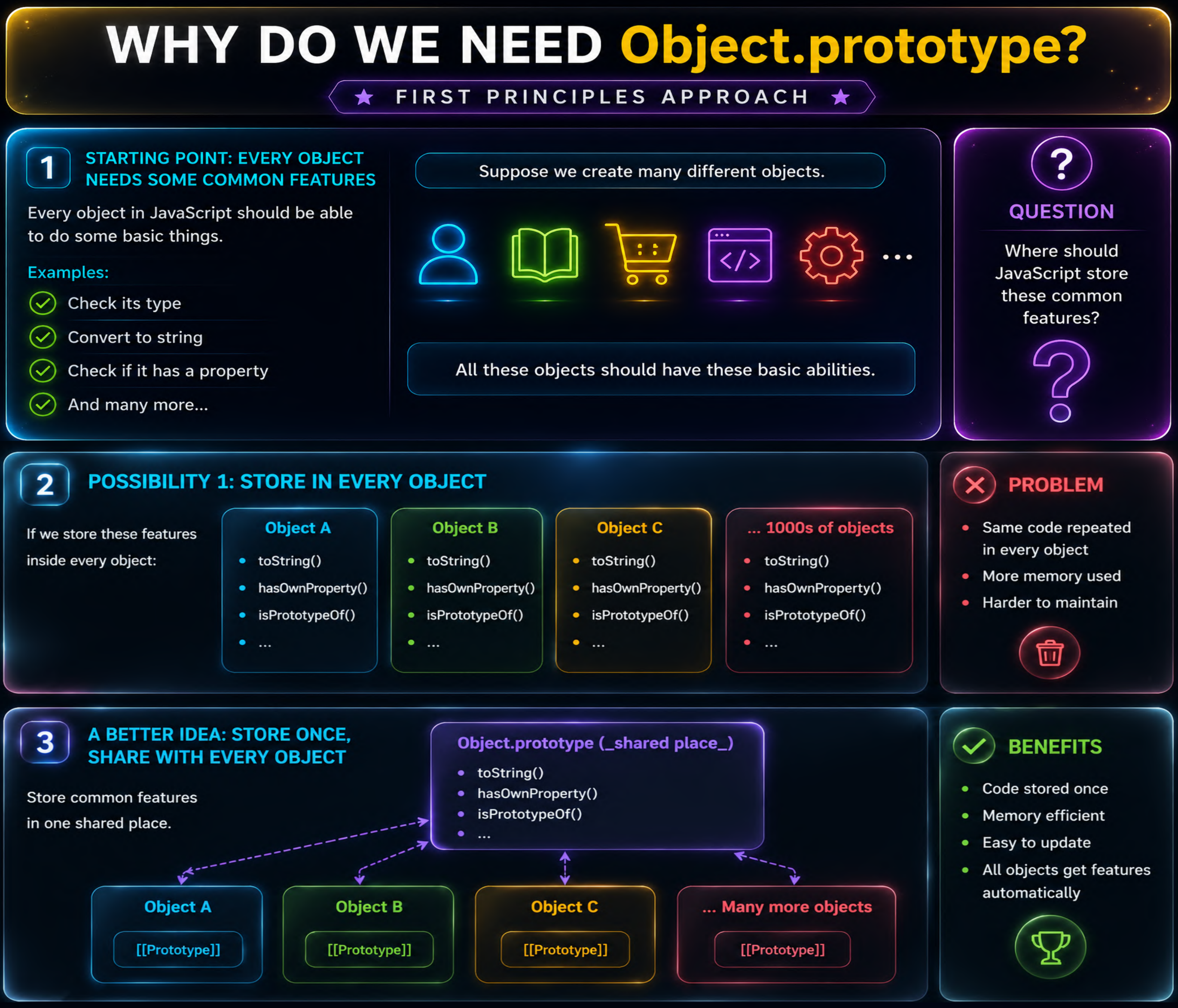
FINAL MENTAL MODEL

After prototypes (optimized): n → Object #1 `[[NumberData]] : 10` → `Number.prototype` (methods stored once) → Efficient. Shared. No waste.



JS BUILT-IN CONSTRUCTORS AND THEIR PROTOTYPES

Number.prototype	String.prototype	Array.prototype	Date.prototype	RegExp.prototype
toString() toFixed() toPrecision() valueOf() ...	charAt() indexOf() toString() toUpperCase() ...	push() pop() shift() unshift() forEach() ...	getTime() getFullYear() toDateString() toISOString() ...	exec() test() toString() compile() ...
↑	↑	↑	↑	↑
new Number() Creates a Number Object 10	**new String()** Creates a String Object “Hello”	**new Array()** Creates an Array Object [1, 2, 3]	**new Date()** Creates a Date Object	**new RegExp()** Creates a RegExp Object /abc/



WHY DO WE NEED Object.prototype?

★ FIRST PRINCIPLES APPROACH ★

1 STARTING POINT: EVERY OBJECT NEEDS SOME COMMON FEATURES

Every object in JavaScript should be able to do some basic things.

Examples:

- ✓ Check its type
- ✓ Convert to string
- ✓ Check if it has a property
- ✓ And many more...

Suppose we create many different objects.

All these objects should have these basic abilities.

QUESTION

Where should JavaScript store these common features?

2 POSSIBILITY 1: STORE IN EVERY OBJECT

If we store these features inside every object:

Object A	Object B	Object C	... 1000s of objects
toString() hasOwnProperty() isPrototypeOf() ...	toString() hasOwnProperty() isPrototypeOf() ...	toString() hasOwnProperty() isPrototypeOf() ...	toString() hasOwnProperty() isPrototypeOf() ...

PROBLEM

- Same code repeated in every object
- More memory used
- Harder to maintain

3 A BETTER IDEA: STORE ONCE, SHARE WITH EVERY OBJECT

Store common features in one shared place.

Object.prototype (_shared place_)

- toString()
- hasOwnProperty()
- isPrototypeOf()
- ...

Object A	Object B	Object C	... Many more objects
[[Prototype]]	[[Prototype]]	[[Prototype]]	[[Prototype]]

BENEFITS

- Code stored once
- Memory efficient
- Easy to update
- All objects get features automatically

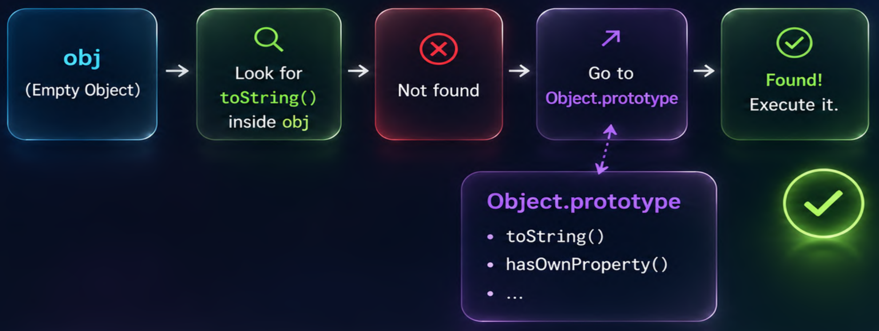
4 HOW IT WORKS: PROTOTYPE CHAIN

When an object tries to access a property:

1. JavaScript looks in the object itself.
2. If not found, it looks in Object.prototype.
3. If found, it uses it.

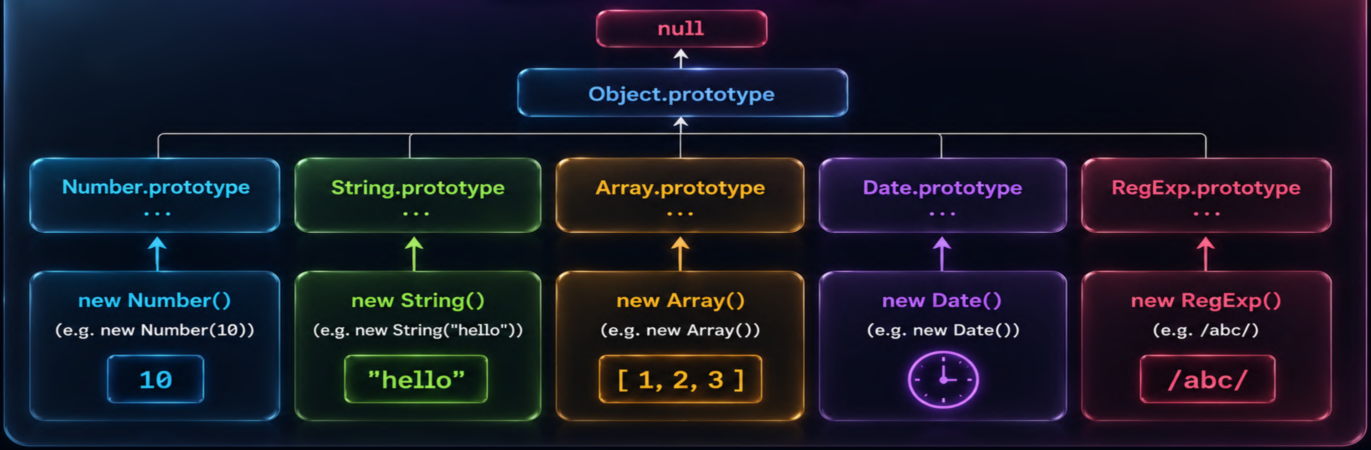
Example:

```
let obj = {};
obj.toString();
```



JS COMPLETE MAJOR PROTOTYPE HIERARCHY IN JAVASCRIPT JS

THE BIG PICTURE



1 NUMBER OBJECT CHAIN

null
↑
Object.prototype
↑
Number.prototype
↑
new Number(10)

Methods on Number.prototype

- toFixed()
- toPrecision()
- toExponential()
- valueOf()
- toString()

2 STRING OBJECT CHAIN

null
↑
Object.prototype
↑
String.prototype
↑
new String("hello")

Methods on String.prototype

- charAt()
- slice()
- substring()
- replace()
- match()
- split()
- trim()

3 ARRAY OBJECT CHAIN

null
↑
Object.prototype
↑
Array.prototype
↑
new Array()

Methods on Array.prototype

- push()
- pop()
- shift()
- unshift()
- map()
- filter()
- reduce()
- find()
- forEach()

4 DATE OBJECT CHAIN

null
↑
Object.prototype
↑
Date.prototype
↑
new Date()

Methods on Date.prototype

- getFullYear()
- getMonth()
- getDate()
- getTime()
- toISOString()

5 REGEXP OBJECT CHAIN

null
↑
Object.prototype
↑
RegExp.prototype
↑
/abc/

Methods on RegExp.prototype

- test()
- exec()
- toString()

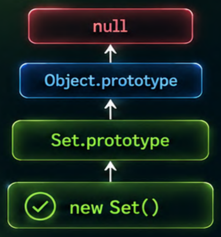
6 MAP OBJECT CHAIN

null
↑
Object.prototype
↑
Map.prototype
↑
new Map()

Methods on Map.prototype

- set()
- get()
- delete()
- clear()
- has()

7 SET OBJECT CHAIN



null

↑

Object.prototype

↑

Set.prototype

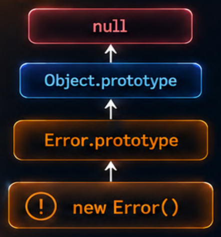
↑

✓ new Set()

Methods on Set.prototype

- add()
- delete()
- clear()
- has()

8 ERROR OBJECT CHAIN



null

↑

Object.prototype

↑

Error.prototype

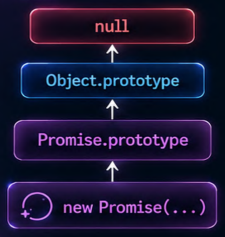
↑

! new Error()

Methods on Error.prototype

- name
- message
- toString()

9 PROMISE OBJECT CHAIN



null

↑

Object.prototype

↑

Promise.prototype

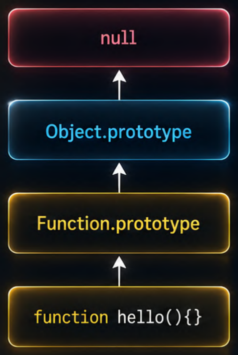
↑

new Promise(...)

Methods on Promise.prototype

- then()
- catch()
- finally()

10 FUNCTION OBJECT CHAIN



null

↑

Object.prototype

↑

Function.prototype

↑

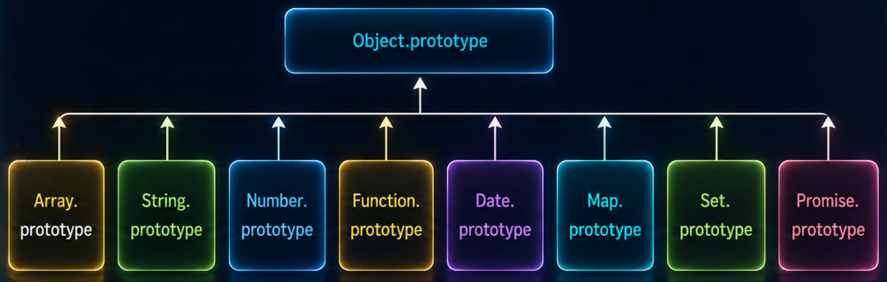
function hello(){}

Methods on Function.prototype

- call()
- apply()
- bind()
- toString()

11 IMPORTANT RELATIONSHIP

Every built-in prototype is itself an object and ultimately inherits from Object.prototype.



Object.prototype

↑

Array.prototype | String.prototype | Number.prototype | Function.prototype | Date.prototype | Map.prototype | Set.prototype | Promise.prototype